Relational Database Model: Table

- In the relational data model, an entity is a relation, or a table

Attributes

Key

* StudentID	StudentName	StudentAddress	StudentYear
1201	Chris Smith	23 William St.	2016
1334	Anne Riley	38 Union Ave.	2017
1432	John Kerry	17 Wood Lane	2016
2874	Doug Dale	6 Lake Rd.	2018
⋮	⋮	⋮	⋮

Row (Tuple)

Value of the attribute

Properties of Relation

- (1) The order of columns of a relation is not important
- (2) The order of row of a relation is not important
- (3) Each **cell** in the table can have only one single value. The value could be "**null**"
- (4) No two records in a relation have the same value of primary key

Candidate Key and Alternative Key

- A relation can have more than one attribute (or a combination of attributes) that can be used to identify the unique record in the table
- For instance, if we assume StudentName (including first name, middle name, and last name) is unique in the database, then the STUDENT relation can have two **candidate keys**: StudentID and StudentName. If the DBA chooses StudentID to be the primary key for STUDENT, then StudentName is called the **alternative key**

Conversion of ER Model to Relational Database Model

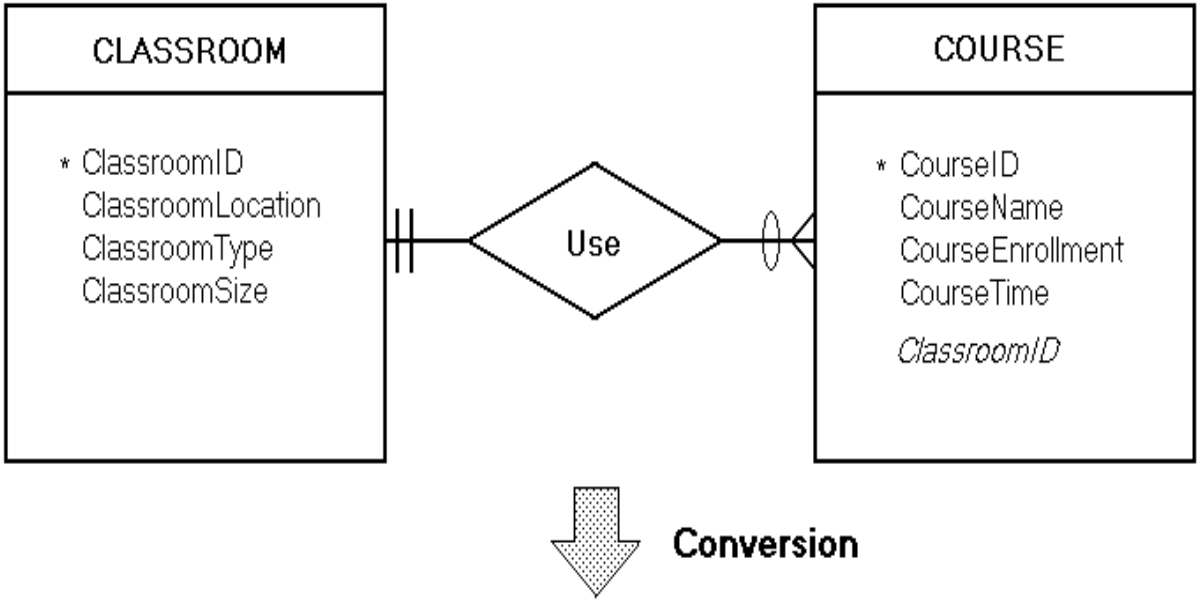
(1)Constructing Tables

(2)Defining Constraints

Constructing Tables

- It is a one seamless step procedure if our "standard" physical ER diagram is used
- (1) An entity in the ER model is a relation in the relational data model
- (2) The primary key of an entity in the ER model is the primary key of the corresponding relation in the relational data model
- (3) The foreign key in an entity in the ER model is an attribute of the corresponding relation in the relational data model

Convert ERD to Relations



CLASSROOM Relation			
* ClassroomID	ClassroomLocation	ClassroomType	ClassroomSize

COURSE Relation				
* CourseID	CourseName	CourseEnrollment	CourseTime	*ClassroomID*

Defining Constraints

- (1) For a relationship with 1:M cardinality, no constraint is needed as it is the default case

- (2) For a relationship with 1:1 cardinality, define the constraint using the following pseudo-code:

  Table {CONSTRAINT *foreign-key* UNIQUE}

  For instance, for the 1:1 relationship between PROFESSOR and OFFICE, the constraint is

  PROFESSOR {CONSTRAINT *OfficeID* UNIQUE}

Defining Constraints (cont'd)

- (3) For a relation with 1 modality, no constraint is needed as it is the default case. Nevertheless, the following pseudo-code can be used to make the default explicit:

  Table {CONSTRAINT MANDATORY TO} The-Other-Table

  For instance, for the 1:1 relationship between PROFESSOR and OFFICE, the following constraint imposes 1 modality to the OFFICE table:

  OFFICE {CONSTRAINT MANDATORY TO} PROFESSOR

- (4) For a relation with 0 modality, define the constraint using the following pseudo-code:

  Table {CONSTRAINT OPTIONAL TO} The-Other-Table

  For instance, for the 1:1 relationship between PROFESSOR and OFFICE, the following constraint imposes 0 modality to the PROFESSOR table:

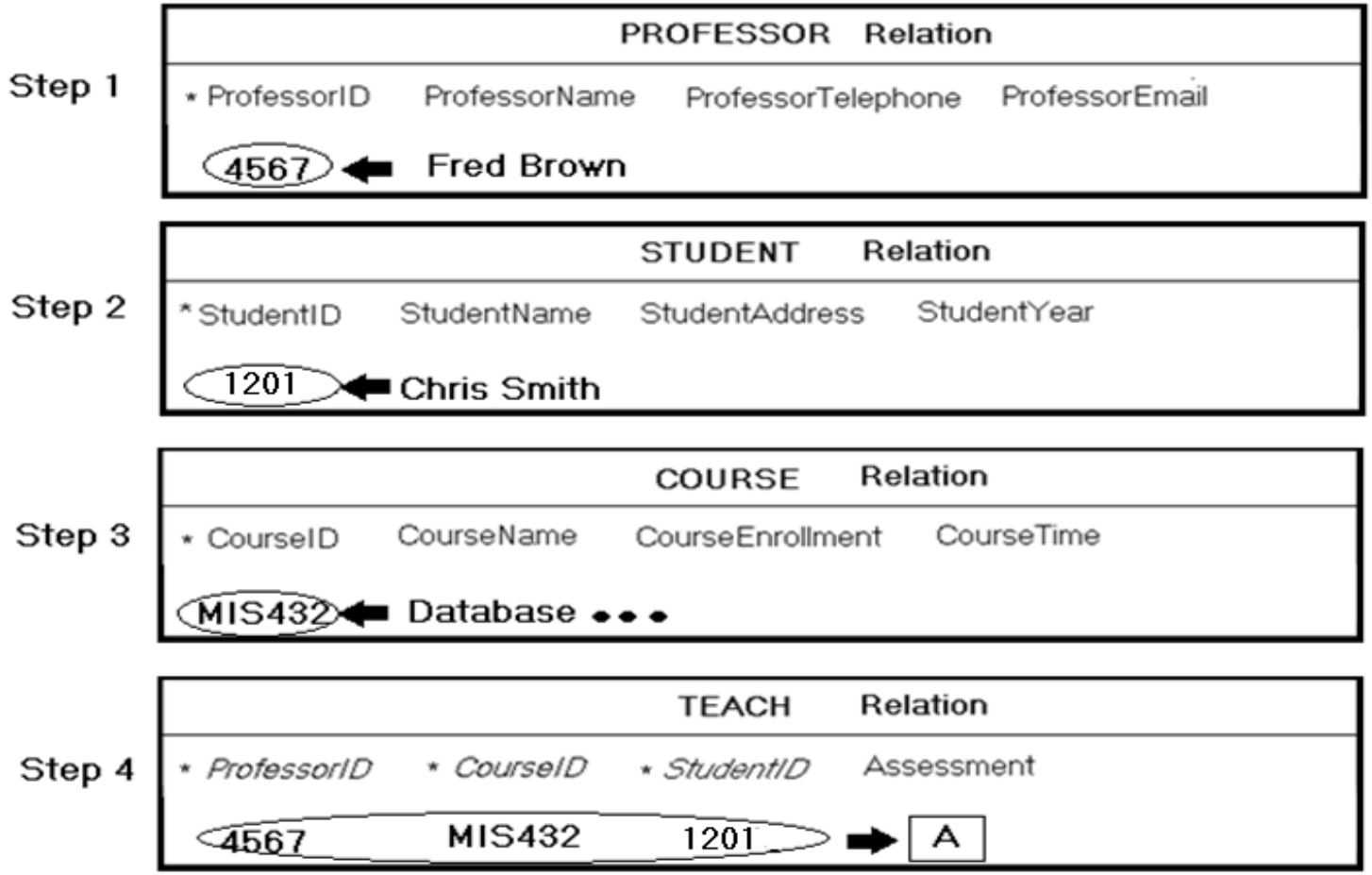
  PROFESSOR {CONSTRAINT OPTIONAL TO} OFFICE

Data Retrieval from Relational Database

- Simple cases: Data retrieval from a single table each time - Trace primary keys and foreign keys to find needed data
- Highly integrated data retrieval – Join multiple tables using keys and foreign keys of the tables

Example: Simple data retrieval

Find the assessment made by Professor Fred Brown for student Chris Smith in the Database Design & Implementation course.



Join Tables

List course ID, course title, course enrollment, course time for each course along with its classroom's data including classroom ID, classroom type, classroom size.

It is impossible to retrieve the needed data using the simple data retrieval approach

CLASSROOM Relation			
* ClassroomID	ClassroomLocation	ClassroomType	ClassroomSize

COURSE Relation				
* CourseID	CourseName	CourseEnrollment	CourseTime	*ClassroomID*

Merged Table

COURSE-CLASSROOM

CourseID	CourseName	CourseEnrollment	CourseTime	**ClassroomID**	ClassroomLocation	ClassroomType	ClassroomSize
MIS212	Programming	40	MWF 9-10	T-001	T-Building	Comp-Lab	50
MIS432	Database	30	MWF 11-12	T-001	T-Building	Comp-Lab	50
ACT211	Accounting I	40	TR 10-11	L-213	L-Building	Lecture	60
FIN312	Finance	35	MWF 9-10	L-213	L-Buldinbg	Lecture	60
MGT211	Organization	38	TR 10-11	K-123	K-Bulding	Case-Room	40

- The merged table contains redundant data
- The merged table provides data integration

Referential Integrity

- As the primary key in the 1-side table is the foreign key in the M-side table, there are potential violations of data integrity
- For example, when the user deletes a classroom records but the classroom has already been assigned to several courses, what should the database do?
- The DBMS controls the data integrity based on different rules

Delete Rule:

- (1) **Restrict –** If there is a record in another table that is relating to the record to be deleted, the deletion is prohibited

CLASSROOM Relation			
* ClassroomID	ClassroomLocation	ClassroomType	ClassroomSize

COURSE Relation				
* CourseID	CourseName	CourseEnrollment	CourseTime	*ClassroomID*

- A classroom record is allowed to be deleted only when it has not been assigned to any course

Delete Rule:

- (2) **Cascade –** All records in another table that are relating to the record to be deleted will be deleted as well

CLASSROOM Relation			
* ClassroomID	ClassroomLocation	ClassroomType	ClassroomSize

COURSE Relation				
* CourseID	CourseName	CourseEnrollment	CourseTime	*ClassroomID*

- If one deletes a classroom record, all records of the courses which use the classroom will be deleted automatically

Delete Rule:

- (3) **Set-to-Null** - All records in another table that are relating to the record to be deleted will be updated by setting the FK attribute to NULL

CLASSROOM Relation			
* ClassroomID	ClassroomLocation	ClassroomType	ClassroomSize

COURSE Relation				
* CourseID	CourseName	CourseEnrollment	CourseTime	*ClassroomID*

- If a classroom record is deleted, all courses that have been assigned to this classroom will be changed to “null” that means “no classroom”

Normalization

- **Normalization** is a process that evaluates the table structures and minimizes data redundancy

Functional Dependency

- **Functional dependency** defines the **determination relationships** between the attributes
- The statement "*B is functionally dependent on A*" or "*A determines B*", where B is an attribute, A is an attribute or a group of attributes, means that "*if you know the value A, then you know the value of B*"

$A \rightarrow B$

Examples of functional dependency

- StudentID $\rightarrow$ StudentName
- [StudentID+CourseID] $\rightarrow$ Grade

Functional Dependency

- The functional dependencies for a database are assumed by the database designer based on her/his conceptual understanding of the relationships between attributes in the real world
- The functional dependencies for the database are usually objective “common sense”, but can also be assumptions which may or may not be totally objective

Normal Form

- A **normal form** represents a certain type of functional dependency property of tables
- We will emphasize on **first normal form** (**1NF**), **second normal form** (**2NF**), and **third normal form** (**3NF**), although there are higher-level normal forms
- Note that (1) if a table in 3NF, it must be also in 2NF; (2) if a table in 2NF, it must be in 1NF as well; (3) the database is in 3NF only if all tables are in 3NF

0NF

- A 0NF table has a missing value or multiple values in a cell, or has not primary key

Student ID	Student Name	Course ID	Grade
1001	Chris Smith	ACT211	A
		MIS315	B
		MIS322	
10021	Anne Riley	ACT212	B
		MGT311	
		FIN312	A
10293	John Kerry	MGT212	B
		ACT211	A

Table with no primary key

Student ID	Student Name	Grade
10001	Chris Smith	A
10001	Chris Smith	B
10001	Chris Smith	B
10021	Anne Riley	B
10021	Anne Riley	A
10021	Anne Riley	A
10293	John Kerry	B
10293	John Kerry	A

Conversion from 0NF to normal form

- ***Step 1:*** Eliminate missing values or repeating values in any cell

- ***Step 2:*** Define all functional dependencies among the attributes based on the business environment or common sense. If there is an attribute without involving functional dependency (e.g., Grade in the previous slide), you must create a functional dependency for it by adding a new attribute(s) (e.g., add CourseID for Grade in the previous slide)

- ***Step 3:*** Identify the primary key for the table (e.g., [StudentID+CourseID]), so that if the value of the primary key is known then the value of any other attribute must be known

1NF

- A table is in **first normal form** (**1NF**) if it meets the following two conditions
- (1) The table has no missing value or multiple values in a cell
- (2) The primary key of the relation is identified based on assumptions of functional dependency among all attributes. These assumptions should be consistent with the **business rules** in the business environment

1NF

* StudentID	StudentName	Major	Department	* CourseID	CourseTitle	Grade
10003	John Smith	ACT	Accounting	ACT211	Fin.Accouting	B
10003	John Smith	ACT	Accounting	ENG101	Intro.English I	A
10014	Anne Gold	CHM	Chemistry	CHM101	Intro.Chemistry	C
10014	Anne Gold	CHM	Chemistry	CHM231	Adv.Chemistry	B
12343	Bob Brown	GNS	Biology	BIO101	Intro.Biology	A
12343	Bob Brown	GNS	Biology	CHM101	Intro.Chemistry	B
12343	Bob Brown	GNS	Biology	BIO345	Genetics II	A
13345	Mike Green	ACT	Accounting	ENG101	Intro.English I	B
13345	Mike Green	ACT	Accounting	ENG201	Intro.English II	B
13345	Mike Green	ACT	Accounting	ENG345	Literature	C

StudentID → StudentName

StudentID → Major

StudentID → Department

CourseID → CourseTitle

Major → Department

StudentID+CourseID → Grade

Data redundancy in 1NF table

- Data redundancy in 1NF

  (e.g., the fact that 1003 is John Smith repeats)

- Data redundancy causes modification anomalies

Data modification anomaly

(1) **Update anomaly**

If a repeated fact is changed, all redundant values must be changed in each place they are stored

(2) **Insertion anomaly**

A table with data redundancy could preclude useful facts

(3) **Deletion anomaly**

A deletion of record from the table can cause unintentional deletion of facts

Data redundancy in 1NF table (cont'd)

- The root of this problem is known as **partial key dependency**
- Student data depend on StudentID only (not [StudentID+CourseID])
- Course data depend on CourseID only (not [StudentID+CourseID]).
- A partial key dependency occurs when the value in a non-key attribute of a table is dependent on the value of a part, but not the whole, of the table's combination primary key

Normalize 1NF table

STUDENT

* StudentID	StudentName	Major	Department
10003	John Smith	ACT	Accounting
10014	Anne Gold	CHM	Chemistry
12343	Bob Brown	GNS	Biology
13345	Mike Green	ACT	Accounting

StudentID → StudentName

StudentID → Major

StudentID → Department

Major → Department

COURSE

* CourseID	CourseTitle
ACT211	Fin.Accouting
CHM101	Intro.Chemistry
CHM231	Adv.Chemistry
BIO101	Intro.Biology
BIO345	Genetics
ENG101	Intro.English I
ENG201	Intro.English II
ENG345	Literature

CourseID → CourseTitle

GRADING

* StudentID	* CourseID	Grade
10003	ACT211	B
10003	ENG101	A
10014	CHM101	C
10014	CHM231	B
12343	BIO101	A
12343	CHM101	B
12343	BIO345	A
13345	ENG101	B
13345	ENG201	B
13345	ENG345	C

StudentID+CourseID → Grade

Steps of normalizing 1NF table

- ***Step 1:*** Make a list of potential individual keys derived from the primary key of the table in 1NF. For the table in 1NF in our example, three potential individual keys can be derived from [StudentID+CourseID]; they are: StudentID, CourseID, and [StudentID+CourseID] itself
- ***Step 2:*** For each attribute in the table in 1NF, assign it to the pertinent functionally dependent potential individual key. In our example, you can assign StudentName to StudentID, CourseTitle to CourseID, Grade to [StudentID+CourseID], and so on. The final result of the two steps is shown in the previous slide
- ***Step 3:*** Revise and preserve the given functional dependencies. A functional dependency might be lost as a result of the normalization process. One must revise and preserve the functional dependencies

Table in 2NF

- The normal form of 1NF table without partial key dependency is called 2NF
- A table in 2NF can still contain data redundancy. For instance, the STUDENT table has repeating facts about major and department
- This problem is caused by **non-key dependency** (or **transitive dependency**)
- A non-key dependency occurs when the value in a non-key attribute of a table is dependent on the value of another non-key attribute

Normalize table in 2NF

STUDENT

* StudentID	StudentName	Major
10003	John Smith	ACT
10014	Anne Gold	CHM
12343	Bob Brown	GNS
13345	Mike Green	ACT

StudentID → StudentName

StudentID → Major

OWNERSHIP

* Major	Department
ACT	Accounting
CHM	Chemistry
GNS	Biology

Major → Department

COURSE

* CourseID	CourseTitle
ACT211	Fin.Accouting
CHM101	Intro.Chemistry
CHM231	Adv.Chemistry
BIO101	Intro.Biology
BIO345	Genetics
ENG101	Intro.English I
ENG201	Intro.English II
ENG345	Literature

CourseID → CourseTitle

GRADING

* StudentID	* CourseID	Grade
10003	ACT211	B
10003	ENG101	A
10014	CHM101	C
10014	CHM231	B
12343	BIO101	A
12343	CHM101	B
12343	BIO345	A
13345	ENG101	B
13345	ENG201	B
13345	ENG345	C

StudentID+CourseID → Grade

- The normal form of a table that is in 2NF and contains no non-key dependency is called third normal form or 3NF

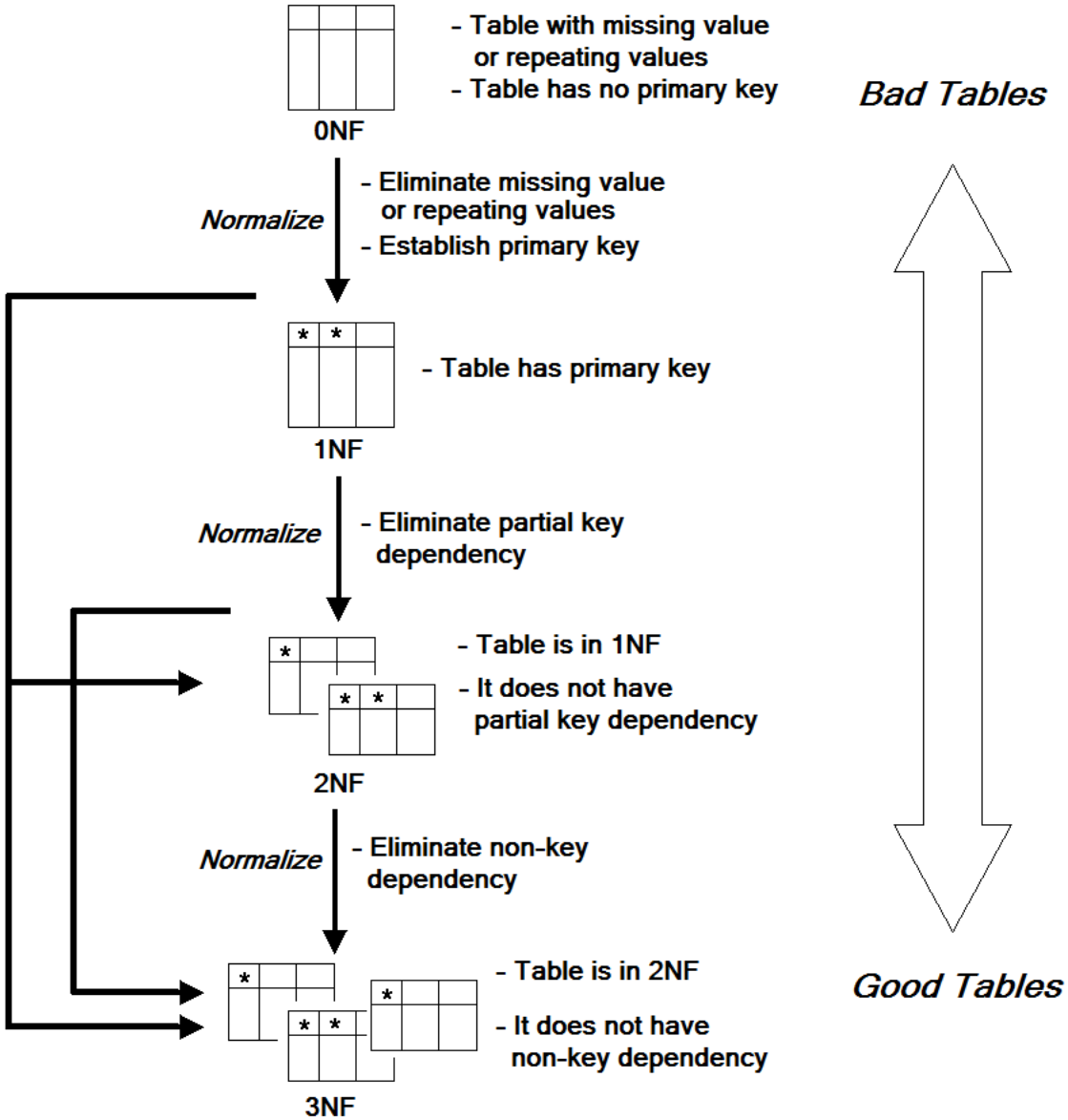
Normalize 2NF into 3NF

- ***Step 1:*** Identify any non-key dependency in the table in 2NF. In our example, Major $\rightarrow$ Department is a non-key dependency
- ***Step 2:*** Create a new table (e.g., OWNERSHIP in our example) for the attributes involved in the non-key dependency (Major and Department in this example), and assign the primary key for this new table (Major in this example)
- ***Step 3:*** Keep the determinant attribute involved in the non-key dependency (Major in our example) in the original table (STUDENT) as the foreign key from the new table (OWNERSHIP), and delete other attributes involved in the non-key dependency (Department in this example) from the original table (STUDENT)
- ***Step 4:*** Revise and preserve the given functional dependencies. A functional dependency might be lost as a result of the normalization process. One must revise and preserve the functional dependencies

Determine normal form of non-0NF table

- 1. If the given table has partial key dependency, then it is in 1NF
- 2. If the given table does not have partial key dependency and has non-key dependency, then it is in 2NF
- 3. If the given table does not have partial key dependency and does not have non-key dependency, then it is in 3NF

The normalization procedure



Higher-level NF

- Tables in 3NF could still have data modification anomalies due to other types of functional dependencies
- **BCNF** (**Boyce-Codd Normal Form**) addresses the “reverse” dependency problem in a 3NF table
- **4NF** addresses multivalued dependency problem
- 5NF and domain-key normal form (DKNF) exist

Requirements on Normalization

- The minimum requirements on normalization would be 3NF in general
- However, many professionals insists that a database must meet 4NF

“Reverse” Dependency in Tables in 3NF

- “Reverse” dependency: a non-candidate-key attribute is the determinant of an attribute of the primary key
- A reverse dependency occurs when the value of a part of the combination key is reversely determined by the value of another non-candidate-key attribute
- This causes redundancy

* StudentID	* Major	Advisor	MajorProject
10014	CHM	Brown	3.5
10014	GNS	Green	3.1
13345	ACT	Smith	3.2
12343	GNS	Green	3.7
13469	ACT	Jones	3.4

StudentID+Major → Advisor Advisor → Major ("Reverse" dependency)

StudentID+Major → MajorProject

Normalize Table in 3NF into BCNF

STUDENT

* StudentID	* Advisor	MajorProject
10014	Brown	3.5
10014	Green	3.1
13345	Smith	3.2
12343	Green	3.7
13469	Jones	3.4

StudentID+Advisor → MajorProject

ADVISOR

* Advisor	Major
Brown	CHM
Green	GNS
Smith	ACT
Jones	ACT

Advisor → Major

Normalize 3NF into BCNF

- ***Step 1:*** Identify any reverse dependency in the table in 3NF. In our example, Advisor $\rightarrow$ Major is a reverse dependency
- ***Step 2:*** Create a new table (e.g., ADVISOR in our example) for the attributes involved in the reverse dependency (Advisor and Major in this example), and assign the primary key for this new table (Advisor in this example)
- ***Step 3:*** Make the determinant attribute involved in the reverse dependency (Advisor in our example) to be a part of the combination primary key of the original table (STUDENT), and delete other attributes involved in the reverse dependency (Major in this example) from the original table (STUDENT)

Multivalued Dependency Problems of Tables in BCNF

- Multivalued dependency problem.

- StudentID $\rightarrow\rightarrow$ Major
  StudentID $\rightarrow\rightarrow$ Minor

  The same facts are repeated
  (e.g., 10014 has MKT major
  10003 has SPA minor)

STUDENT

* StudentID	* Major	* Minor
10003	MKT	SPA
10003	FIN	SPA
10014	MKT	FRA
10014	MKT	GER
23540	CHE	BUS
35462	ENG	BUS

StudentID $\rightarrow\rightarrow$ Major

StudentID $\rightarrow\rightarrow$ Minor

- A multivalued dependency occurs when the table is in BCNF and the value of an attribute determines a set of more than one possible value of the other two attributes in the table

Normalize Table in BCNF into 4NF

MAJORS

* StudentID	* Major
10003	MKT
10003	FIN
10014	MKT
23540	CHE
35462	ENG

StudentID + Major →

MINORS

* StudentID	* Minor
10003	SPA
10014	FRA
10014	GER
23540	BUS
35462	BUS

StudentID + Minor →

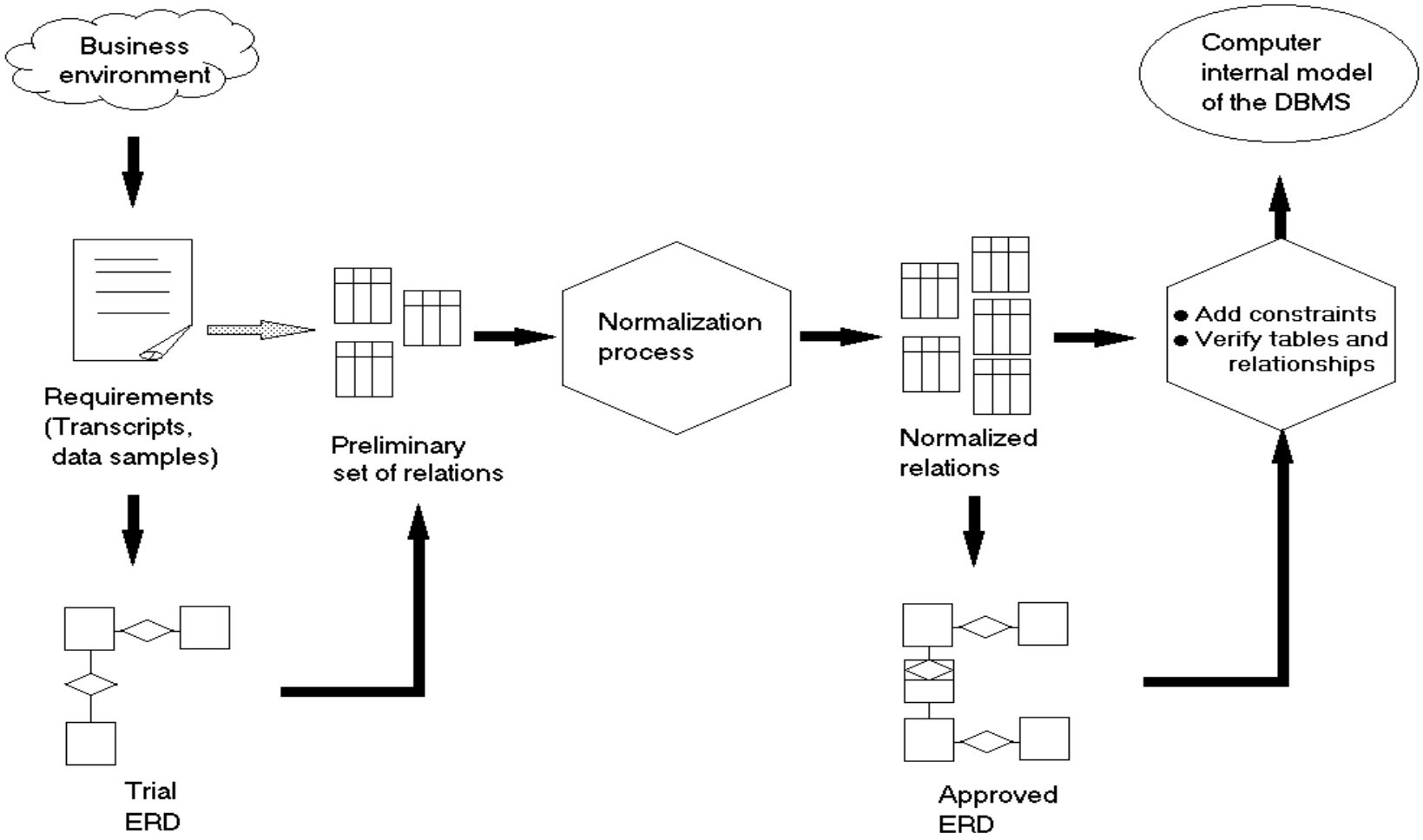
Normalize BCNF into 4NF

- ***Step 1:*** Identify any MVD in the table which is in BCNF. In our example, StudentID $\rightarrow\rightarrow$ Major and StudentID $\rightarrow\rightarrow$ Minor are MVD
- ***Step 2:*** Create a new table (e.g., MAJORS in our example) for the attributes involved in the MVD (StudentID and Major in this example), and assign the combination primary key for this new table ([StudentID+Major] in this example)
- ***Step 3:*** Delete the multivalued attribute involved in the MVD (i.e., the right hand side of the MVD, Major in this example) from the original table (STUDENT in this example), and rename the table (e.g., to MINORS in this example)
- ***Step 4:*** Repeat Step 1 through Step 3 for the table resulted in Step 3 (e.g., MINORS in this example) until no MVD exists in all resulting tables

Logical Database Design

- **Logical database design** is the process that defines the databases structures to meet the data requirements of the business
- ***Step 1.*** Generate an initial ER model
- ***Step 2.*** Convert the initial ER model into a preliminary set of tables
- ***Step 3.*** Perform normalization on the preliminary set of tables into normalized tables (see the procedure in the previous section)
- ***Step 4.*** Revise the ER model based on the normalized tables
- ***Step 5.*** Convert the normalized tables into the internal model for a selected DBMS (e.g., Oracle, DB2, Access, etc.)
- ***Step 6.*** Attach pertinent constraints to applicable tables
- ***Step 7.*** Confirm that the internal model for the selected DBMS is consistent with the ER model

Logical Database Design



Three Important Points

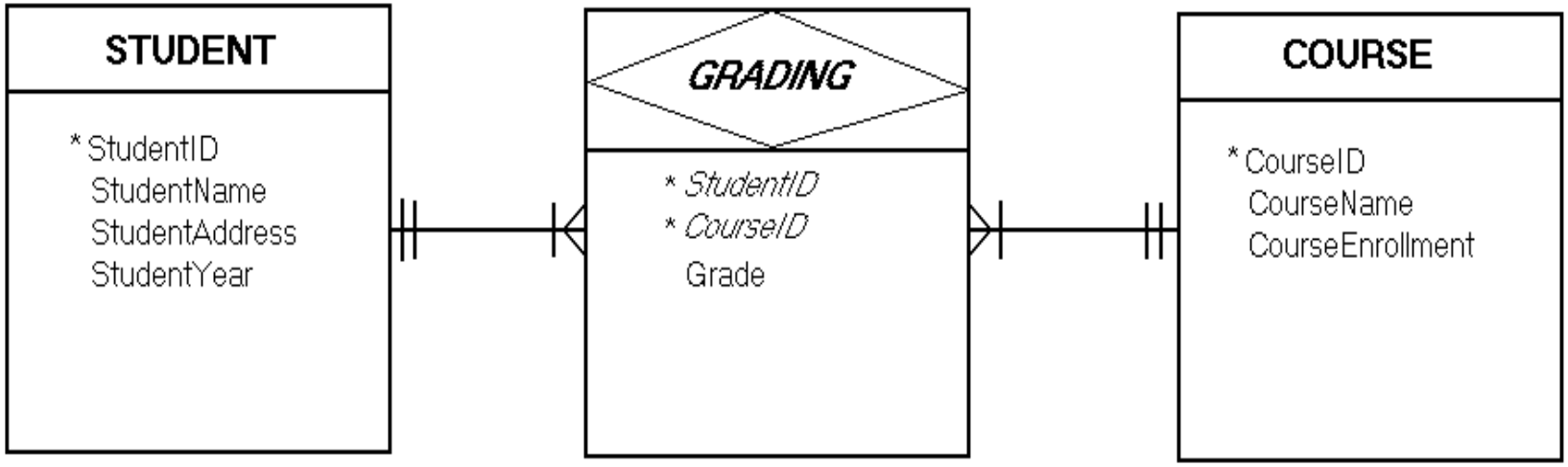
- (1) The logical database design for a database is an iterative process.
- (2) The ERD of a database is a blueprint for the database. It is used for human communication. More importantly, because it is difficult for humans to verify whether the computer internal model is consistent with those normalized tables without a blueprint, the ERD must be used.
- (3) Logical database design ensures the data needs for the business, but does not take the performance of the database into account. The ultimate consequence of data in higher-level normal forms is the inefficiency of data retrieval. To solve this problem in a large database, physical database design must be applied in addition to logical database design.

SQL

- **SQL (Structured Query Language)** is a universal language for querying and updating databases
- SQL integrates features of data definition languages (DDL) and data manipulation languages (DML)
- When using other computer languages (such as COBOL, C, Java, ADO.NET) to develop business process software that is connected to databases (Oracle, DB2, etc.), one must write SQL code

SQL Examples

- The toy database for the examples



```
CREATE TABLE tblStudent
  (StudentID CHAR(8),
   StudentName CHAR(20),
   StudentAddress CHAR(20),
   StudentEnrolYear INT,
   PRIMARY KEY (StudentID));
```

Listing 6.1. CREATE TABLE

```
DROP TABLE tblStudent;
```

Listing 6.2. DROP TABLE

```
INSERT INTO tblStudent
 VALUES ('01234567', 'John',
         '285 Westport', 2018);
```

Listing 6.3. INSERT a Record

```
UPDATE tblStudent
  SET StudentAddress = '300 Eastport'
  WHERE StudentID='01234567';
```

Listing 6.4. UPDATE a Record

```
DELETE FROM tblStudent
  WHERE StudentID='01234567';
```

Listing 6.5. DELETE a Record